Journal de développement — Freelance Automation

Session 1 — Définition du projet et cadrage initial

Ce que je voulais faire

Avant même de coder, je voulais partir sur une base claire. Mon objectif était de définir un projet cohérent et structuré, sans partir dans tous les sens.
J'ai donc décidé de demander à l'IA de m'aider à construire une vision globale du projet, avec :

- une architecture
- des fonctionnalités
- un pipeline complet

Je voulais éviter le piège de coder sans plan.

Comment j'ai travaillé avec l'IA

Prompt utilisé (approximatif) :

"Aide-moi à définir un projet complet d'automatisation de candidatures freelance avec backend, frontend et IA, avec une architecture claire et évolutive."

L'IA m'a aidé à :

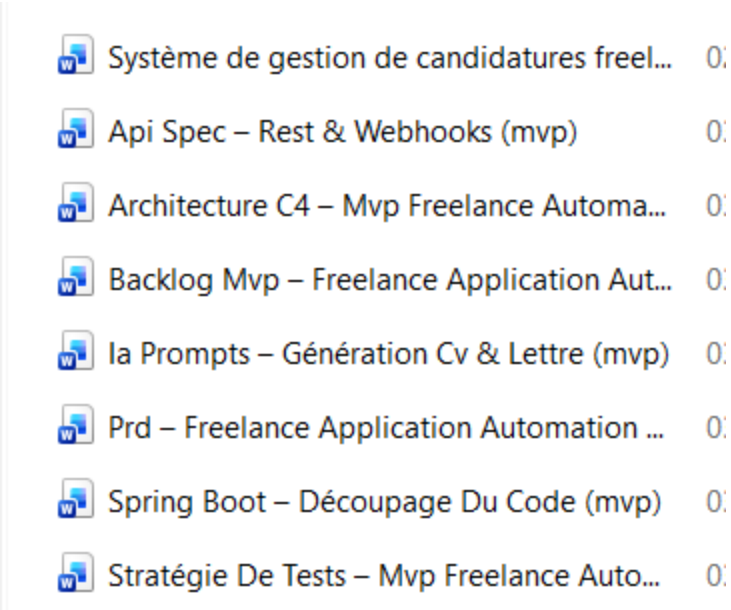
- structurer le projet (auth, CV, jobs, matching, candidature...)
- découper en modules
- proposer une architecture backend + frontend

Ce qui a bien marché :

- Très bonne capacité à structurer le projet globalement
- Donne une vision claire rapidement

Ce que j'ai dû corriger :

- L'IA propose souvent des architectures trop ambitieuses pour un MVP
- J'ai accepté trop facilement certaines complexités



Le fait que j'ai commencé par ceci m'a beaucoup aidé à ne se perdre en aucun moment et à facilement garder le contexte et là ou je suis pour moi et pour n'importe quel autre LLM.

Ce qui a bloqué ou changé

Rien de bloquant techniquement, mais une erreur importante :

J'ai commencé directement avec une vision trop "scalable"
→ micro-services implicites
→ séparation poussée
→ logique "production-ready"

Alors que je n'avais encore rien de fonctionnel.

Et malgré ceci je ne le regrette pas surtout que ce projet m a beaucoup plus et j'envisage en aller jusqu'au bout même si je pense qu'il y a déjà des SaaS concurrents qui font la même chose mais ce me vas.

Ce que j'en retiens

- L'IA est excellente pour structurer un projet
- Mais je l'ai forcé vers des architectures complexes

- J’aurais dû simplifier dès le départ

Erreur que j’ai déjà faite (hackathon) et que je reproduis :

Vouloir faire “propre et scalable” avant d’avoir un produit qui marche

Mais par contre je sais que je vais la refaire.

Remarques libres

Je pense que j’ai voulu “bien faire” trop tôt.
Avec du recul, j’aurais dû viser :

“un truc moche mais fonctionnel” dès le début

Session 2 — Choix du backend et premier regret structurant

Ce que je voulais faire

Une fois le cadre posé, je voulais passer à quelque chose de concret : faire exister le backend, avec une base suffisamment solide pour brancher progressivement tout le pipeline.

Mon objectif était simple :

- gérer l’authentification,
- stocker les données,
- exposer des endpoints REST,
- construire la structure du projet.

Je voulais quelque chose de sérieux, lisible, modulaire, et je voulais poser une base qui me donne confiance pour la suite.

Comment j’ai travaillé avec l’IA

J’ai demandé à l’IA de m’aider à construire un backend Spring Boot avec :

- JWT,
- sécurité,

- structure controller/service/repository,
- persistance avec PostgreSQL,
- organisation cohérente par modules.

Je l'ai utilisée comme un accélérateur de boilerplate, mais aussi comme un guide de structure. Typiquement, je lui demandais :

- comment organiser les packages,
- quoi mettre dans config,
- quoi isoler dans auth, user, profile, cv, etc.,
- comment ne pas trop casser la logique modulaire dès le départ.

Ce qui a bien marché

Là-dessus, l'IA a été très utile :

- elle m'a fait gagner beaucoup de temps,
- elle a rendu le démarrage beaucoup plus propre,
- elle m'a aidé à garder une structure lisible,
- elle a bien géré tout ce qui est répétitif et standard.

Le backend a pris une forme cohérente assez vite, ce qui m'a donné l'impression de construire quelque chose de "sérieux".

Ce qui a bloqué ou changé

Le vrai problème ici n'a pas été technique, mais décisionnel : **j'ai choisi Java / Spring Boot alors que ce n'était probablement pas la meilleure techno pour ce projet-là**.

Avec du recul, je pense que FastAPI aurait été beaucoup plus adapté pour plusieurs raisons :

- le projet est fortement orienté IA,
- je suis plus rapide en Python,
- le prototypage aurait été plus simple,
- l'intégration avec les modèles et les traitements texte aurait été plus naturelle,
- la friction générale aurait été plus faible.

J’ai choisi Spring Boot parce que je voulais un backend “solide”, “propre”, “professionnel”, presque dans une logique d’image ou de sérieux technique. Mais un projet n’a pas besoin d’une techno impressionnante ; il a besoin d’une techno adaptée.

Ce que j’en retiens

Cette séance m’a rappelé une chose importante : l’IA peut proposer du bon code, mais elle ne choisit pas la bonne stratégie à ma place. Le choix de techno reste mon choix, donc mon erreur quand il est mauvais.

Si je devais refaire ce projet aujourd’hui, je prendrais probablement FastAPI dès le départ pour le MVP, quitte à changer plus tard si nécessaire. Là, j’ai privilégié une forme de rigueur technique au détriment de la vitesse et du confort d’exécution.

Remarques libres

Je pense que j’ai voulu faire un projet “d’ingénieur logiciel sérieux” alors que j’aurais dû faire un projet “efficace, rapide à itérer, facile à débloquer”. C’est une erreur que je reconnais assez clairement maintenant.

Session 3 — Authentification, profil et premiers signaux rassurants

Ce que je voulais faire

Après la mise en place du backend, je voulais sécuriser un premier vrai parcours utilisateur :

- inscription,
- connexion,
- récupération d’un token,
- accès à une route protégée,
- création / mise à jour du profil freelance.

Je voulais valider que le backend ne soit pas juste “une structure”, mais un système qui sait déjà gérer un vrai utilisateur.

Comment j’ai travaillé avec l’IA

J’ai utilisé l’IA de manière très dirigée, presque par petites tranches :

- créer les DTO d’auth,
- générer le service JWT,
- brancher le filtre d’authentification,
- sécuriser les routes,
- créer les endpoints /register, /login, /me,
- puis brancher le profil.

À chaque étape, je lui demandais quelque chose de très précis, puis je testais immédiatement.

Ce qui m’a aidé, c’est que je ne lui demandais pas “fais tout”, mais plutôt “ajoute ce morceau sans casser ce qui marche déjà”.

Ce qui a bien marché

Cette partie s’est bien déroulée. Le login, le JWT, le profil, tout ça a fini par fonctionner et ça m’a donné un premier sentiment de progression réelle.

Le fait de pouvoir tester avec PowerShell, récupérer un token, appeler /api/me, puis mettre à jour un profil m’a rassuré. J’avais enfin quelque chose qui ressemblait à un vrai backend applicatif.

Ce qui a bloqué ou changé

Il y a eu plein de petites frictions :

- problèmes de packages,
- chemins de fichiers,
- erreurs de compilation,
- soucis autour de Maven,
- confusion autour des endpoints protégés,

- erreurs 403/404/400 au mauvais moment.

Ce n'étaient pas des blocages conceptuels, mais une accumulation de petits frottements techniques. Ce qui m'a marqué, c'est que l'IA aide à corriger vite, mais elle donne parfois des instructions très propres "en théorie", alors que dans mon environnement réel, il fallait sans cesse ajuster les chemins, les noms de classes, les commandes Windows, etc.

Ce que j'en retiens

Cette phase m'a appris que l'IA est très efficace pour accélérer la mise en place d'un backend standard, mais qu'il faut rester très concret :

- vérifier les chemins,
- adapter à Windows,
- tester immédiatement,
- ne jamais supposer que "si ça compile dans l'exemple, ça compile chez moi".

Remarques libres

C'est à partir de là que j'ai commencé à sentir que le projet avançait vraiment. C'était encore loin de l'objectif final, mais j'avais une base utilisateur réelle.

Session 4 — Le parsing de CV : la partie la plus frustrante du backend

Ce que je voulais faire

Je voulais intégrer une brique indispensable au projet : le CV.

L'objectif était de permettre :

- l'upload du fichier,
- son stockage,
- l'extraction de texte,
- et l'utilisation future de ce texte pour le matching et la génération.

En théorie, ça paraissait simple. En pratique, c’est probablement la partie qui m’a le plus frustré.

Comment j’ai travaillé avec l’IA

J’ai demandé à l’IA des choses très ciblées :

- comment parser un PDF en Java,
- quelle librairie utiliser,
- comment gérer les problèmes d’encodage,
- comment tester l’extraction,
- comment structurer le service CV.

Elle m’a proposé des solutions classiques, notamment PDFBox, puis m’a aidé à brancher :

- l’entité CV,
- le stockage,
- l’extraction du texte,
- un endpoint pour lire le texte extrait.

Ce qui a bien marché

Le pipeline technique a fini par fonctionner :

- upload OK,
- stockage OK,
- lecture du texte extrait OK.

En ce sens, la logique backend est bien en place.

Ce qui a bloqué ou changé

Le gros problème a été l'encodage et l'affichage. Je recevais du texte extrait, mais avec des caractères cassés :

- accents illisibles,
- apostrophes bizarres,
- rendu parfois incohérent.

Et le plus embêtant, c'est que je testais surtout avec curl et PowerShell. Sans frontend, je voyais très mal ce qui se passait réellement. Je recevais des réponses, mais je ne savais pas toujours si le problème venait :

- de l'extraction,
- de l'encodage,
- de l'affichage console,
- ou du stockage en base.

J'ai essayé plusieurs corrections :

- modifier le parsing,
- manipuler l'encodage,
- réparer le texte après extraction,
- comparer différentes sorties.

Mais ça m'a coûté pas mal de temps.

Ce que j'en retiens

Cette séance m'a appris plusieurs choses.

D'abord, le parsing PDF est plus compliqué que ce qu'on imagine, surtout dès qu'on veut un résultat propre.

Ensuite, un backend sans feedback visuel rend le débogage beaucoup plus difficile. Je crois que j'ai perdu du temps ici parce que je voulais résoudre trop de choses "côté moteur" alors que je n'avais pas d'interface simple pour constater ce qui sortait réellement.

Enfin, j'ai compris que pour un MVP, il faut parfois accepter un résultat "suffisamment exploitable" au lieu de poursuivre une perfection technique qui bloque tout le reste.

Remarques libres

C'est clairement une des zones où j'ai le plus stagné. Je ne considère pas cette partie comme élégante. Elle fonctionne, mais elle m'a rappelé qu'un problème qui paraît local peut ralentir tout un projet si on s'acharne dessus sans visibilité.

Session 5 — Ingestion des offres et naissance du vrai pipeline métier

Ce que je voulais faire

Une fois le profil et le CV en place, je voulais connecter l'autre moitié du projet : les offres.

Mon objectif était de faire exister la logique suivante :

- une offre arrive dans le système,
- elle est stockée,
- on évite les doublons,
- puis on peut la mettre en relation avec le profil et le CV.

Je n'avais pas encore branché n8n pour de vrai, mais je voulais que le backend soit prêt à recevoir ses données.

Comment j'ai travaillé avec l'IA

J'ai demandé à l'IA de me guider pour créer :

- une entité d'offre,
- un DTO webhook,
- un repository,
- un service d'ingestion,

- un endpoint webhook,
- une logique de déduplication simple.

Là encore, je l'ai utilisée comme un partenaire de progression incrémentale : ajouter cette feature, la tester, puis seulement passer à la suivante.

Ce qui a bien marché

Cette partie a été plutôt fluide :

- le webhook a marché,
- la persistance a marché,
- la déduplication a marché.

J'ai trouvé ça très satisfaisant, parce que le projet commençait enfin à relier ses deux sources principales :

- le candidat,
- l'offre.

Ce qui a bloqué ou changé

Le blocage principal n'était pas dans le code, mais dans ma réflexion. À ce moment-là, je me suis dit assez clairement que le CV seul n'a pas vraiment de valeur sans l'offre, et que l'offre seule n'a pas vraiment de valeur sans le CV. C'est la mise en relation des deux qui donne du sens au produit.

C'est aussi à ce moment-là que j'ai commencé à voir plus clairement les limites de certains choix "théoriques" faits au début : tant que le pipeline métier n'existe pas, l'architecture ne veut pas dire grand-chose.

Ce que j'en retiens

Le projet a commencé à devenir concret le jour où les offres ont pu entrer proprement dans le système.

J'en retiens aussi que construire les briques séparées est utile, mais qu'à un moment il faut absolument les relier. Sinon on a juste une collection de modules, pas un produit.

Remarques libres

C’est à partir de là que j’ai commencé à avoir envie de brancher n8n pour de vrai. Je voyais beaucoup mieux ce qu’il me manquerait pour passer d’une simulation à un vrai flux automatisé.

Session 6 — Décision de faire le frontend

Ce que je voulais faire

Créer une interface pour :

- voir les données
- tester les endpoints
- rendre le projet concret

Comment j’ai travaillé avec l’IA

Prompt :

“Structure-moi un frontend Angular standalone avec routing, features modules et services pour consommer mon backend”

Ce qui a bien marché

- Structure Angular claire :
  - features/
  - core/
  - services
- Routing lazy loading
- Intégration rapide avec backend

Ce qui a bloqué

Problèmes techniques Angular :

- imports incorrects
- composants non trouvés
- erreurs de routing
- problème avec zone.js

Exemple :

NG0908: Angular requires Zone.js

Ce que j'ai fait

- debug avec l'IA
- correction progressive
- ajout de zone.js
- restructuration des imports

Ce que j'en retiens

- Angular est puissant mais fragile au setup
- L'IA aide beaucoup mais nécessite validation constante
- Les erreurs sont souvent liées à :
    - structure
    - chemins
    - config

Remarques libres

Même si le front est moche, il est **extrêmement utile**
→ c'est ce qui a débloqué tout le reste

Session 7 — Génération IA : le projet devient enfin ce que j'avais en tête

Ce que je voulais faire

Je voulais brancher la génération de contenu avec l'IA pour produire :

- un CV adapté à une offre,
- une lettre de motivation contextualisée.

C'était un moment important, parce que cette partie correspond exactement au cœur de l'idée qui m'intéressait.

Comment j'ai travaillé avec l'IA

J'avais déjà préparé des prompts en amont dans mes documents de cadrage. Je m'en suis servi comme référence pour structurer la génération :

- pas d'invention de compétences,
- pas d'expérience fictive,
- reformulation uniquement,
- ton professionnel,
- génération conditionnée par le CV et l'offre.

L'IA m'a aidé à :

- préparer un mock d'abord,
- puis intégrer Mistral,
- configurer la clé API,
- construire les appels,
- et faire remonter les réponses dans le backend.

Ce qui a bien marché

Le fait d'avoir commencé par un mock a été une bonne idée. Ça m'a permis de valider d'abord le pipeline, sans tout casser avec l'appel réel à un LLM.

Ensuite, quand Mistral a été branché, j'ai enfin vu le projet prendre une forme qui ressemblait à mon intuition initiale : non plus juste une API, mais un outil capable de transformer des données en candidature contextualisée.

Ce qui a bloqué ou changé

Le principal blocage n'a pas été dans la logique métier, mais dans la configuration :

- variable d'environnement,
- .env,
- démarrage cassé si la clé n'était pas disponible,
- difficulté à comprendre d'où venait exactement l'erreur.

J'ai aussi été confronté au problème de l'encodage dans les textes générés, hérité en partie du parsing du CV. Ça a dégradé la qualité visuelle du résultat, même si le fond restait globalement exploitable.

Ce que j'en retiens

Cette séance m'a appris qu'il faut distinguer :

- la logique d'orchestration,
- la logique de génération,
- la qualité des données d'entrée.

Même avec un bon LLM, si le texte de départ est mal encodé ou imparfait, le résultat final en souffre.

Mais malgré ça, je considère que c'est une des séances les plus importantes, parce que c'est là que le projet a vraiment commencé à ressembler à ce que j'avais envie de construire.

Remarques libres

J’ai pris plaisir à cette étape, même si elle était imparfaite. C’est à ce moment-là que le projet m’a le plus motivé.

Session 8 — Pipeline complet fonctionnel

Ce que je voulais faire

Avoir un flow complet :

CV → Jobs → Matching → Génération → Validation → Soumission

Ce que je voulais faire

Je voulais brancher la génération de contenu avec l’IA pour produire :

- un CV adapté à une offre,
- une lettre de motivation contextualisée.

C’était un moment important, parce que cette partie correspond exactement au cœur de l’idée qui m’intéressait.

Comment j’ai travaillé avec l’IA

J’avais déjà préparé des prompts en amont dans mes documents de cadrage. Je m’en suis servi comme référence pour structurer la génération :

- pas d’invention de compétences,
- pas d’expérience fictive,
- reformulation uniquement,
- ton professionnel,
- génération conditionnée par le CV et l’offre.

L’IA m’a aidé à :

- préparer un mock d’abord,
- puis intégrer Mistral,
- configurer la clé API,
- construire les appels,
- et faire remonter les réponses dans le backend.

Ce qui a bien marché

Le fait d’avoir commencé par un mock a été une bonne idée. Ça m’a permis de valider d’abord le pipeline, sans tout casser avec l’appel réel à un LLM.

Ensuite, quand Mistral a été branché, j’ai enfin vu le projet prendre une forme qui ressemblait à mon intuition initiale : non plus juste une API, mais un outil capable de transformer des données en candidature contextualisée.

Ce qui a bloqué ou changé

Le principal blocage n’a pas été dans la logique métier, mais dans la configuration :

- variable d’environnement,
- .env,
- démarrage cassé si la clé n’était pas disponible,
- difficulté à comprendre d’où venait exactement l’erreur.

J’ai aussi été confronté au problème de l’encodage dans les textes générés, hérité en partie du parsing du CV. Ça a dégradé la qualité visuelle du résultat, même si le fond restait globalement exploitable.

Ce que j’en retiens

Cette séance m’a appris qu’il faut distinguer :

- la logique d’orchestration,
- la logique de génération,
- la qualité des données d’entrée.

Même avec un bon LLM, si le texte de départ est mal encodé ou imparfait, le résultat final en souffre.

Mais malgré ça, je considère que c’est une des séances les plus importantes, parce que c’est là que le projet a vraiment commencé à ressembler à ce que j’avais envie de construire.

Remarques libres

J’ai pris plaisir à cette étape, même si elle était imparfaite. C’est à ce moment-là que le projet m’a le plus motivé.

Comment j’ai travaillé avec l’IA

J’ai utilisé l’IA pour :

- créer les entités,
- ajouter les services,
- brancher les déclencheurs métier,
- exposer les endpoints de lecture.

Mais à ce stade, l’intérêt principal de l’IA n’était plus seulement de générer du code. Elle m’a surtout aidé à garder le cap, à savoir ce qui avait déjà été fait, ce qui restait, et dans quel ordre avancer.

Ce qui a bien marché

Le backend est devenu cohérent de bout en bout :

- actions auditées,
- notifications créées,
- état métier visible,
- logique complète.

C'est aussi à ce moment-là que j'ai réalisé que, côté backend, j'avais en fait construit une version déjà assez riche du MVP.

Ce qui a bloqué ou changé

Le vrai point ici n'est pas technique, mais réflexif.

Je me suis rendu compte que je reproduisais une erreur que j'avais déjà faite lors d'un hackathon : chercher très tôt à concevoir quelque chose de largement scalable, modulaire et propre, au lieu de viser d'abord un MVP qui me satisfasse pleinement.

Je pense que cette tendance vient du fait que le projet m'intéresse vraiment. Comme l'idée me plaît, j'ai naturellement envie de penser long terme, V2, évolutivité, MCP, orchestration, etc. Mais justement, quand un sujet me plaît, je dois encore plus faire attention à ne pas me disperser.

Ce que j'en retiens

Le projet m'a appris quelque chose de très important sur ma manière de travailler avec l'IA :

L'IA ne simplifie pas la difficulté. Elle **accélère** la direction que je prends.
Donc :

- si ma direction est bonne, elle me fait gagner énormément de temps,
- si ma direction est mauvaise, elle m'aide à aller très vite dans quelque chose de trop compliqué.

J'en retiens aussi que je dois mieux distinguer :

- ce qui est utile maintenant,

- ce qui est intéressant mais prématuré.

Remarques libres

J'ai aussi essayé de forcer un peu la question de **MCP**, surtout parce que je voulais me familiariser avec. Mais je reconnais que, pour ce MVP précis, ce n'était pas réellement nécessaire. C'était davantage une curiosité technique et une envie d'apprendre qu'un besoin produit immédiat.

Je ne regrette pas cet intérêt, parce qu'il s'inscrit dans quelque chose que j'explorais déjà avant même ce cours, avec mes idées autour des MCP, des CLI et de l'orchestration. En revanche, je reconnais clairement que ce n'était pas la priorité du MVP.

Conclusion personnelle

Ce projet m'a appris plusieurs choses que je trouve importantes.

D'abord, travailler avec l'IA n'est pas "plus facile" au sens naïf du terme. Ce n'est pas un raccourci magique. C'est un accélérateur. Elle va plus vite que moi sur :

- la structure,
- le boilerplate,
- la reformulation,
- l'assemblage.

Mais elle m'oblige aussi à être plus lucide sur mes choix. Si je pars dans une mauvaise direction, elle m'aide à produire rapidement quelque chose de sophistiqué mais pas forcément pertinent.

Ensuite, je me suis confirmé que j'ai tendance à vouloir penser "architecture sérieuse" avant "MVP satisfaisant". C'est une qualité dans certains contextes, mais ici ça m'a aussi ralenti. Je l'avais déjà vécu ailleurs, et je l'ai revécu ici.

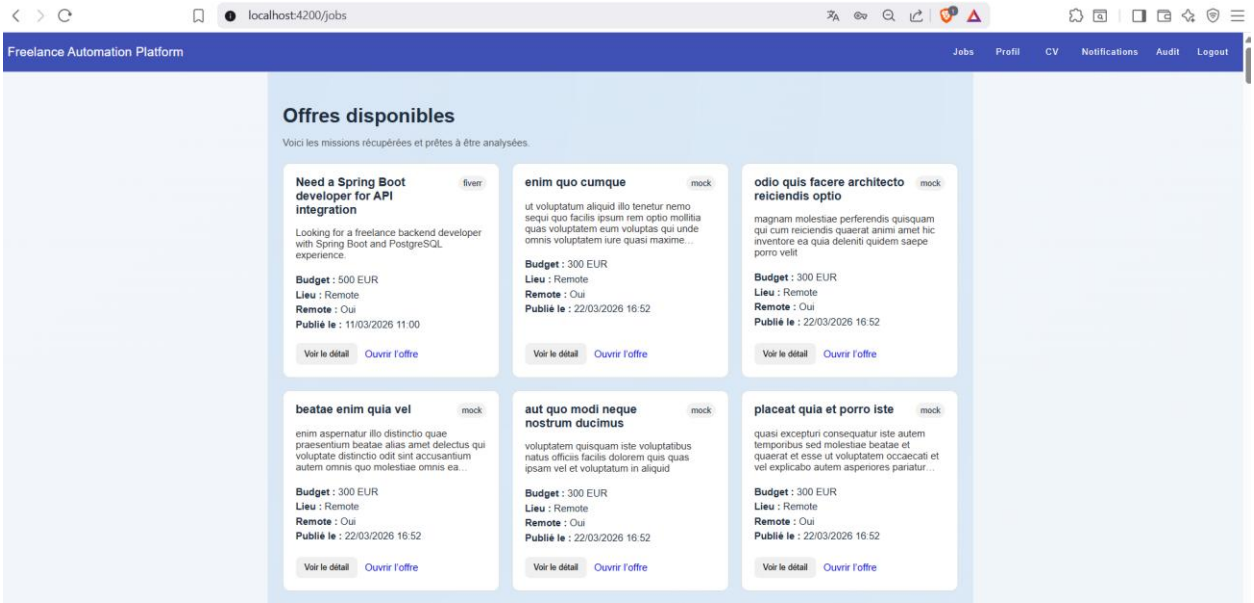
Enfin, ce projet m'a montré que ce que je préfère, ce n'est pas juste coder une API ou brancher une interface, mais construire un système qui a une logique métier forte, qui automatise réellement quelque chose d'utile, et qui garde une place pour la décision humaine.

Je ne considère pas ce projet comme terminé dans ma tête. Au contraire, j'ai l'impression d'avoir construit une première version qui me donne envie d'aller plus loin :

- brancher n8n réellement,
- améliorer le matching,
- améliorer la qualité du parsing,
- renforcer le frontend,
- et peut-être plus tard explorer MCP et CLI dans une version plus mature.

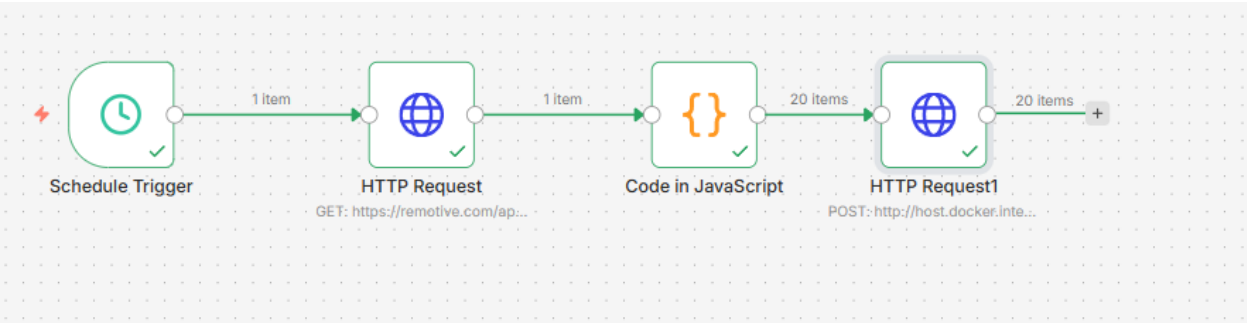
Ce que je retiens surtout, c'est qu'un système imparfait mais complet m'apprend beaucoup plus qu'un système théoriquement parfait qui n'existe pas encore.

Session 9 – Amélioration du front et première intégration de n8n pour scraping



Cette séance a marqué le passage d'un système principalement testé manuellement à un produit alimenté automatiquement par une source externe. Le plus important n'a pas été le code n8n lui-même, mais le travail de normalisation et de compatibilité avec mon backend existant. J'ai dû faire attention à ne pas casser mon format d'ingestion, notamment sur budget, postedAt et la description HTML de Remotive. J'ai aussi compris qu'une fois le pipeline vivant, la priorité change : ce n'est plus d'ajouter des fonctionnalités, mais de stabiliser la fiabilité et la qualité des données.

Ainsi j'ai trouver que ca sera difficile pour l'instant de scrapper à partir de Fiverr à cause de leur robot.txt du coup j'ai passé sur Remotive qui offre un Access simple et documenté par api, même sans authentification.



Sachant que je n'ai toujours pas fini ce que je prévois encore qui sera d'intégrer un MCP et limiter le stockage dans la bd puisque je n'aurai pas besoin des offres qui datent de plus de 3 jours par exemple.

Ainsi plus de nodes dans mon n8n et via MCP pour candidater automatiquement et rendre le cv et la motivation en doc téléchargeable sous une format ATS friendly ainsi qu' ajouter la méthode adéquate de candidature avec d'autres model d LLM payantes pour essayer de monétiser mon SaaS (comme le système de ILovePdf par exemple).

Et merci pour cette opportunité.